

COGNOME:

NOME:

MATRICOLA:

Prova Parziale di Algoritmi e Strutture Dati (9CFU), 16 Aprile 2025

Esercizio 1 [5 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva? Quando non indicata, la base di un logaritmo è 2.

(A) $\log n$, $\log(n^{4n})$, $n^{3/2} + n \log(\sqrt{n})$, $4^{n/2}$, $\sum_{i=0}^n 3^i$

(B) $(\log n)^{2025}$, $\sqrt[3]{n}$, $n^{\log_3 5}$, $n \log n!$, $\sum_{i=0}^{\log n} 2^i$

(C) $\frac{n}{\log n}$, $\sqrt{n} \log n$, $\log n!$, $\binom{n}{2}$, $\binom{n}{n-5}$

(D) Nessuna delle altre

Esercizio 2 [5 punti]

Si considerino i seguenti algoritmi in cui l'input è un intero positivo n potenza di 2. Si determini il tempo di esecuzione nel caso peggiore di ALGO2.

Algorithm ALGO1(n)

1: $i = 1$	$i = 1$	$j \rightarrow 2$	① 2	1 passo	$\text{Totale} = 1 + 2 + 3 + \dots + n =$ $= \sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$ $= \frac{1}{2}n^2 + \frac{1}{2}n \Rightarrow \Theta(n^2)$
2: while $i \leq n$ do	$i = 2$	$j \rightarrow 4$	① 2 ③ 4	2 passi	
3: $j = 1$	$i = 3$	$j \rightarrow 6$	① 2 ③ 4 ⑤ 6	3 passi	
4: while $j < 2i$ do					
5: $j = j + 2$					
6: end while					
7: $i = i + 1$					
8: end while	$i = n$	$j \rightarrow 2n$		i passi	

Algorithm ALGO2(n)

1: if $n > 1$ then	
2: ALGO1($n/4$)	$\rightarrow \Theta\left(\left(\frac{n}{4}\right)^2\right) = \Theta\left(\frac{n^2}{16}\right) = \Theta(n^2)$
3: $i = 0$	
4: while $i \leq 2025\sqrt{n}$ do	$i \rightarrow 2025\sqrt{n}$
5: $i = i + 2025$	$i \rightarrow i + 2025$
6: end while	num. iterazioni è $\frac{2025\sqrt{n}}{2025} = \sqrt{n} = \Theta(\sqrt{n})$
7: ALGO2($n/2$)	
8: ALGO2($n/2$)	
9: ALGO2($n/2$)	
10: ALGO2($n/2$)	
11: end if	

$\Theta(n^2) + \Theta(\sqrt{n}) = \Theta(n^2)$
 $T(n) = 4T(n/2) + \Theta(n^2)$

MASTER THM.: $a=4$ $b=2$ $f(n)=n^2$

(A) $\Theta(n^2)$

(B) $\Theta(n \log n)$

(C) $\Theta(n^2 \log n)$

(D) Nessuna delle altre

$n^{\log_2 4} = n^2 = f(n) \rightarrow \text{caso 2}$

$\hookrightarrow T(n) = \Theta(n^2 \log n)$

Esercizio 3 [5 punti]

Quale delle seguenti affermazioni è vera in merito all'algoritmo QUICKSORT?

- (A) La fase Divide viene eseguita in $O(1)$.
- (B) Il tempo di esecuzione nel caso peggiore di QUICKSORT è $O(n \log n)$.
- ☒ (C) Per array in input ordinati in modo decrescente, il numero di confronti effettuati da QUICKSORT è $\Theta(n^2)$.
- (D) Nessuna delle altre.

Esercizio 4 [5 punti]

Quale delle seguenti strutture dati permette di implementare in modo più efficiente, nel *caso peggiore*, l'operazione SEARCH in un dizionario?

- ☒ (A) Array ordinato. $\Rightarrow$ Ricerca binaria $O(\log n)$
- (B) Lista doppiamente concatenata. $\Rightarrow$ scotto uno per uno $O(n)$
- (C) Tavola hash con concatenamento. $\Rightarrow$ non nel caso peggiore
- (D) Lista semplicemente concatenata. $\Rightarrow$ scotto uno per uno $O(n)$

Esercizio 5 [5 punti]

Quale delle seguenti affermazioni è vera?

- (A) Nessuna delle altre.
- (B) Un qualsiasi algoritmo di ordinamento che riceve in input un array già ordinato restituisce in output l'array stesso non modificato.
- ☒ (C) Supponiamo che un array A di n interi distinti soddisfi la seguente proprietà: Ciascun elemento di A è fuori posto di al più $\log n$ posizioni rispetto alla posizione che avrebbe se A fosse ordinato. Allora INSERTION-SORT permette di ordinare A in $O(n \log n)$. *n element: $O(\log n)$*
- (D) Supponiamo di riuscire a modificare l'algoritmo di Karatsuba per moltiplicare due binari x e y di n bit ciascuno nel modo seguente. Dividiamo in modo bilanciato sia x che y in 3 parti (al posto delle 2 di Karatsuba) e calcoliamo ricorsivamente 6 moltiplicazioni (al posto delle 3 di Karatsuba). Il nuovo algoritmo è più efficiente di quello di Karatsuba.

SBARRAMENTO - 15 punti sugli esercizi precedenti

Esercizio 6 [10 punti] Sia $A[1..n]$ un array di n interi che soddisfa le seguenti condizioni:

- (i) $|A[i+1] - A[i]| \leq 1$, per ogni $i \in \{1, 2, \dots, n-1\}$;
- (ii) $A[1] \leq 0$ e $A[n] > 0$.

Un indice k tale che $A[k] = 0$ è detto *zero* dell'array A .

1. Dato un array A di $n \geq 2$ interi che soddisfa (i) e (ii), dimostrare che A ha almeno uno zero. [3 pt]
2. Fornire lo pseudo-codice di un algoritmo Divide et Impera che trova uno zero di A in $O(\log n)$. [4 pt]
3. Dimostrare la correttezza dell'algoritmo fornito. [3 pt]

Esercizio 1 [5 punti]

Quale delle seguenti sequenze di funzioni è tale che ogni funzione è O della successiva? Quando non indicata, la base di un logaritmo è 2.

(A) $\log n$, $\log(n^{4n})$, $n^{3/2} + n \log(\sqrt{n})$, $4^{n/2}$, $\sum_{i=0}^n 3^i$

(B) $(\log n)^{2025}$, $\sqrt[3]{n}$, $n^{\log_3 5}$, $n \log n!$, $\sum_{i=0}^{\log n} 2^i$

(C) $\frac{n}{\log n}$, $\sqrt{n} \log n$, $\log n!$, $\binom{n}{2}$, $\binom{n}{n-5}$

(D) Nessuna delle altre

A. $\log n \leq \log(n^{4n}) \leq n^{3/2} \leq 4^{n/2} \leq \sum_{i=0}^n 3^i$

• $\log(n^{4n}) = 4n \log n$

• $n^{3/2} + n \log(\sqrt{n}) = n^{3/2} + \frac{1}{2} n \log n$

• $4^{n/2} = 4^{1/2 \cdot n} = 2^n$

• $\sum_{i=0}^n 3^i = 3^{n+1}$

B. $(\log n)^{2025} \leq \sqrt[3]{n} \leq n^{\log_3 5} \leq n \log n! > \sum_{i=0}^{\log n} 2^i$

• $\sqrt[3]{n} = n^{1/3} \approx n^{0.33}$

• $n^{\log_3 5} \approx n^{1.464}$

• $n \log n! = n \cdot n \log n = n^2 \log n$

• $\sum_{i=0}^{\log n} 2^i = 2^{\log n + 1} = 2 \cdot n$

C. $\frac{n}{\log n} > \sqrt{n} \log n$

• $\frac{n}{\log n} = n \cdot \frac{1}{\log n} = n (\log n)^{-1}$

• $\sqrt{n} \log n = n^{1/2} \log n$

Esercizio 6 [10 punti] Sia $A[1..n]$ un array di n interi che soddisfa le seguenti condizioni:

(i) $|A[i+1] - A[i]| \leq 1$, per ogni $i \in \{1, 2, \dots, n-1\}$;

(ii) $A[1] \leq 0$ e $A[n] > 0$.

Un indice k tale che $A[k] = 0$ è detto *zero* dell'array A .

1. Dato un array A di $n \geq 2$ interi che soddisfa (i) e (ii), dimostrare che A ha almeno uno zero. [3 pt]

2. Fornire lo pseudo-codice di un algoritmo Divide et Impera che trova uno zero di A in $O(\log n)$. [4 pt]

3. Dimostrare la correttezza dell'algoritmo fornito. [3 pt]

1. Sappiamo che $A[1] \leq 0$ e $A[n] > 0$

→ La condizione (i) $A[i+1] - A[i] \leq 1 \Rightarrow$ il valore dell'array può variare, tra un elemento e il successivo, solo di $1, 0, -1$.

Supponiamo per assurdo che non esista alcuno zero. Per passare da un valore ≤ -1 a un valore ≥ 1 , l'incremento minimo necessario sarebbe $1 - (-1) = 2$.

Poiché l'incremento massimo consentito è $+1$, l'algoritmo non può saltare lo zero senza toccarlo, Quindi deve esistere almeno un indice k t.c. $A[k] = 0$



2. TrovaZero(A, p, r)

// A ha un solo elemento

if $A[p] == 0$ return p

$q = \lfloor \frac{p+r}{2} \rfloor$

if $A[q] == 0$ return q

else if $A[q] > 0$ TrovaZero(A, p, q) // lo zero è nella metà a sinistra

else TrovaZero($A, q+1, r$) // lo zero si trova nella metà a destra

3. Ad ogni chiamata TrovaZero(A, k, z), l'intervallo $[k, z]$ mantiene la proprietà $A[k] \leq 0$ e $A[z] > 0 \rightarrow$ IP (ii), finché questa proprietà è vera, esiste almeno uno zero nell'intervallo (punto 1).

• Alla prima chiamata $A[1] \leq 0$ e $A[n] > 0$ (IP ii)

• se $A[q] > 0$, l'algoritmo seleziona $[p, q]$. Poiché $A[p] \leq 0$ e $A[q] > 0$.

• se $A[q] < 0$, l'algoritmo seleziona $[q, r]$. Poiché $A[q] < 0$ e $A[r] > 0$.